🚀 Python Mastery Series: Complete Collection

--- Post_01_Introduction.md ---

🚀 Python Mastery Series - Topic 1: Introduction! 🐍

Hey network! 👋 Today I'm sharing some sample code on **Introduction** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
import math,random,string,os
import datetime,time,threading
import json,csv,pygame,requests
import sys
#from PyQt5.QtWidgets import QApplication, QMainWindow
#from PyQt5.QtWidgets import QIcon
#from PyQt5.QtGui import QIcon

#  This is a sample Python script.
# Welcome to Python programming!
print("Welcome to Python programming!")
# Variable declaration = A container for storing data values (string , inte
#--------------------------------- 10 Sep 2025 ----------------------------

#String
first_name = "Vinodh"
food = "Biryani"
email = "vinodh@example.com"
print(f"Hello, {first_name}!")
print(f"I love {food}!")
print(f"My email is {email}.")

# Integer
age = 45
```

```
print(f"I am {age} years old.")

# Float
height = 5.9
print(f"My height is {height} feet.")

# Boolean
is_student = False
print(f"Am I a student? {is_student}")
# Conditional statement
if is_student:
    print("Yes, I am a student.")
else:
    print("No, I am not a student.")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_02_Typecasting___the_p.md ---

🚀 Python Mastery Series - Topic 2: Typecasting = The Process Of Converting A Value From One Data Type To Another Data Type! 🐍

Hey network! 👋 Today I'm sharing some sample code on **Typecasting = The Process Of Converting A Value From One Data Type To Another Data Type** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Typecasting  = the process of converting a value from one data type to an
# Example of typecasting Str(),int(),float(),bool()

name = "Vinodh"
age = 45
```

```
height = 5.9
is_student = False
print(type(name))  # Output: <class 'str'>
print(type(age))   # Output: <class 'str'>
print(type(height))  # Output: <class 'str'>
print(type(is_student))  # Output: <class 'str'>
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_03_Typecasting.md ---

🚀 **Python Mastery Series - Topic 3: Typecasting!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Typecasting** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Typecasting
age = str(age)
print(type(age))  # Output: <class 'str'>
age = int(age)
age +=1
print(type(age))  # Output: <class 'int'>
height = float(height)
is_student = bool(is_student)

# input function = A function that allows user input and returns it as a st
name = input("Enter your name: ")
age = input("Enter your age: ")
print(f"Hello,{name}!")
age = int(age)  # Typecasting input value to integer
```

```
age = age +1
print(f"You are {age} years old.")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_04_Typecasting_input_va.md ---

🚀 **Python Mastery Series - Topic 4: Typecasting Input Values!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Typecasting Input Values** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Typecasting input values

# Rectangle area calculation
length = input("Enter the length of the rectangle: ")
width = input("Enter the width of the rectangle: ")
length = float(length)  # Typecasting input value to float
width = float(width)  # Typecasting input value to float
area = length * width
print(f"The area of the rectangle is: {area} CM²")

# Excerise 2 : Shopping Cart Application
# 1. Add item to cart
# 2. View cart
item = input("Enter the item you want to add to the cart: ")
price = float(input("Enter the price: "))
quantity = int(input("Enter the quantity: "))
total = price * quantity
print(f"{quantity} {item}(s) added to the cart. Total price: ${total:.2f}")
```

```
# Madlib Game
# A word game where you create a story by filling in the blanks with differ
adjective1 = input("Enter an adjective: ")
noun1 = input("Enter a noun: ")
adjective2 = input("Enter an adjective: ")
verb1 = input("Enter a verb ending with 'ing': ")
adjective3 = input("Enter an adjective: ")
print("Here is your Madlib story:")
print(f"Once upon a time, there was a {adjective1} {noun1} who loved {verb1
print(f"One day, the {noun1} met a {adjective2} friend.")
print(f"They decided to go on an adventure to find a {adjective3} treasure.
print("The end!")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_05_Arithmetic___math_Op.md ---

🚀 Python Mastery Series - Topic 5: Arithmetic & Math Operators! 🐍

Hey network! 👏 Today I'm sharing some sample code on **Arithmetic & Math Operators** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Arithmetic & math Operators
# + , - , * , / , % , ** , //
friends = 10
friends = friends + 2  # Addition
friends += 1  # Addition
friends = friends - 2  # Subtraction
friends -= 1  # Subtraction
friends = friends * 2  # Multiplication
```

```
friends *= 2  # Multiplication
friends = friends / 2  # Division
friends /= 2  # Division
friends = friends % 3  # Modulus
friends %= 3  # Modulus
friends = friends ** 2  # Exponentiation
friends **= 2  # Exponentiation
print(friends)  # Final value
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_06_Math_module.md ---

🚀 **Python Mastery Series - Topic 6: Math Module!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Math Module** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Math module
x=3.24
y=-6
z=16
result = round(x)  # Rounds to the nearest integer
result = abs(y)  # Absolute value
result = pow(2, 3)  # 2 raised to the power of 3
result = max(x,y,z)  # Maximum value
result = min(x,y,z)  # Minimum value
print(result)  # Output: 10

print(math.pi)  # Output: 3.141592653589793
print(math.e)   # Output: 2.718281828459045
```

```
result = math.sqrt(6)  # Square root
result = math.ceil(3.2)  # Rounds up to the nearest integer
result = math.floor(3.7)  # Rounds down to the nearest integer
result = math.factorial(9)  # Factorial of 9
print(result)  # Output: 362880

radius = float(input("Enter the radius of the circle: "))
area = math.pi * radius ** 2
print(f"The area of the circle is: {area:.2f} CM²")
circumference = 2 * math.pi * radius
print(f"The circumference of the circle is: {circumference:.2f} CM")

a = float(input("Enter the first number: "))
b = float(input("Enter the second number: "))
c = (pow(a,2) + pow(b,2))**0.5
d= math.sqrt(pow(a,2) + pow(b,2))
print(f"The length of the hypotenuse is: {c}")
print(f"The length of the hypotenuse is: {d}")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_07_if_conditional_state.md ---

🚀 **Python Mastery Series - Topic 7: If Conditional Statements (If, Elif, Else)!** 🐍

Hey network! 👏 Today I'm sharing some sample code on **If Conditional Statements (If, Elif, Else)** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# if conditional statements (if, elif, else)
# 1. if statement
```

```
age = int(input("Enter your age: "))
if age >= 18 and age < 100:
    print("You are eligible to vote.")
elif age <= 0:
    print("Invalid age entered.")
elif age >= 100:
    print("You are eligible for a senior citizen discount.")
else:
    print("You are not eligible to vote.")

response = input("Do you want to continue? (y/n): ").lower()
if response == "y":
    print("You chose to continue.")
elif response == "n":
    print("You chose not to continue.")
#--------------------------------- 11 October 2025 ------------------------
# Python Calculator Application
print("Welcome to the Python Calculator!")
num1 = float(input("Enter the first number: "))
char = input("Enter an operator (+, -, *, /, %, **, //): ")
num2 = float(input("Enter the second number: "))
print("Result:")
if char == "+":
    print(num1 + num2)
elif char == "-":
    print(num1 - num2)
elif char == "*":
    print(num1 * num2)
elif char == "/":
    print(num1 / num2)
elif char == "%":
    print(num1 % num2)
elif char == "**":
    print(num1 ** num2)
elif char == "//":
    print(num1 // num2)
```

```
else:
    print("Invalid operator!")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_08_Logical_Operators_an.md ---

🚀 **Python Mastery Series - Topic 8: Logical Operators And Or Not!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Logical Operators And Or Not** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Logical Operators and or not
# and = True if both statements are true
# or = True if at least one statement is true
# not = Reverses the boolean value
temp =25;
is_raining = True
if temp > 20 and not is_raining:
    print("It's a nice day for a walk.")
elif temp > 30 or is_raining:
    print("It's either too hot or raining, stay indoors.")
else:
    print("The weather is moderate, dress accordingly.")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_09_format_specifiers.md ---

🚀 **Python Mastery Series - Topic 9: Format Specifiers!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Format Specifiers** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# format specifiers
name = "Vinodh"
age = 45
height = 5.9
is_student = False
print("Name: %s" % name)  # String
print("Age: %d" % age)    # Integer

print("Height: %.1f" % height)  # Float with 1 decimal place
print("Height: %.2f" % height)  # Float with 2 decimal places
print("Height: %.3f" % height)  # Float with 3 decimal places
print("Height: %.4f" % height)  # Float with 4 decimal places
print("Am I a student? %s" % is_student)  # Boolean
print("Am I a student? %r" % is_student)  # Boolean (using %r for represent
print("Hello, %s! You are %d years old." % (name, age))  # Multiple values
print("Hello, %s! You are %d years old and %.1f feet tall." % (name, age, h
# f-strings (formatted string literals)
print(f"Hello, {name}! You are {age} years old.")
print(f"Hello, {name}! You are {age} years old and {height:.1f} feet tall."
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_10_String_methods.md ---

🚀 **Python Mastery Series - Topic 10: String Methods!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **String Methods** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# String methods
text = "  Hello, World! Welcome to Python programming.  "
print(text.lower())  # Convert to lowercase
print(text.upper())  # Convert to uppercase
print(text.strip())  # Remove leading and trailing whitespace
print(text.replace("World", "Universe"))  # Replace substring
print(text.split())  # Split string into a list of words
print(text.find("Python"))  # Find substring index
print(text.count("o"))  # Count occurrences of a substring
print(text.startswith("  Hello"))  # Check if string starts with a substrin
print(text.endswith("programming.  "))  # Check if string ends with a subst
print(len(text))  # Get the length of the string
print(text.index("Python"))  # Get the index of a substring (raises error i
print(text.isalpha())  # Check if all characters are alphabetic
print(text.isdigit())  # Check if all characters are digits
print(text.isspace())  # Check if all characters are whitespace
print(text.capitalize())  # Capitalize the first character
print(text.title())  # Capitalize the first character of each word
print(text.center(50))  # Center the string within a specified width
print(text.ljust(50))  # Left-justify the string within a specified width
print(text.rjust(50))  # Right-justify the string within a specified width
print(text.zfill(50))  # Pad the string with zeros on the left to a specifi

# String indexing and slicing
```

```
sample = "Hello, World!"
print(sample[0])    # First character
print(sample[-1])   # Last character
print(sample[0:5])  # Substring from index 0 to 4
print(sample[7:])   # Substring from index 7 to end
print(sample[:5])   # Substring from start to index 4
print(sample[::2])  # Every second character
print(sample[::-1]) # Reverse the string
# List = A collection which is ordered and changeable. Allows duplicate mem
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")  # Add an item to the end of the list
fruits.insert(1, "kiwi")  # Insert an item at a specific position
print(fruits)  # Output: ['apple', 'kiwi', 'banana', 'cherry', 'orange']
print(fruits[0])  # Output: apple
print(fruits[-1])  # Output: orange
print(fruits[1:3])  # Output: ['kiwi', 'banana']
fruits.remove("banana")  # Remove an item by value
fruits.pop()  # Remove the last item
print(fruits)  # Output: ['apple', 'kiwi', 'cherry']
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_11_while_loop___as_long.md ---

🚀 Python Mastery Series - Topic 11: While Loop - As Long As A Condition Is True, It Will Execute A Block Of Code Repeatedly! 🐍

Hey network! 👋 Today I'm sharing some sample code on **While Loop - As Long As A Condition Is True, It Will Execute A Block Of Code Repeatedly** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# while loop - as long as a condition is true, it will execute a block of c
#
# while condition:
#     # code to be executed
name = input("Enter your name: ")
while name == "":
    print("Name cannot be empty. Please enter your name.")
    name = input("Enter your name: ")
print(f"Hello, {name}!")

age = input("Enter your age: ")

while not age.isdigit() or int(age) <= 0:
    print("Invalid age entered. Please enter a valid age.")
    age = input("Enter your age: ")

print(f"Nanditha are {age} years old.")

food = input("Enter your favorite food (q to quit): ")
while not food =="q":
    print(f"{food} is a great choice!")
    food = input("Enter your favorite food (q to quit): ")
print("Goodbye!")

num = int(input("Enter a positive integer (0 to 10): "))
while num <0 or num >10:
    print("Invalid input. Please enter a number between 0 and 10.")
    num = int(input("Enter a positive integer (0 to 10): "))
print(f"You entered: {num}")

# Python compund interest calculator
principal =0;
rate =0;
time =0;
amount =0;
compound_interest =0;
while principal <=0:
```

```
    principal = float(input("Enter the principal amount (greater than 0): "
    if principal <=0:
        print("Principal amount must be greater than 0. Please try again.")

print(f"Principal amount: {principal}")
while rate <=0:
    rate = float(input("Enter the interest rate (greater than 0): "))
    if rate <=0:
        print("Interest rate must be greater than 0. Please try again.")

print(f"Interest rate: {rate}")
while time <=0:
    time = int(input("Enter the time in years (greater than 0): "))
    if time <=0:
        print("Time must be greater than 0. Please try again.")

print(f"Time in years: {time}")

# Calculate compound interest
amount = principal * pow((1 + rate / 100), time)
compound_interest = amount - principal

print(f"Compound interest: {compound_interest}")
print(f"Total amount (principal + interest): {amount}")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_12_For_Loops___used_to_.md ---

🚀 Python Mastery Series - Topic 12: For Loops - Used To Iterate Over A Sequence (Like A List, Tuple, Dictionary, Set, Or String)! 🐍

Hey network! 👏 Today I'm sharing some sample code on **For Loops - Used To Iterate Over A Sequence (Like A List, Tuple, Dictionary, Set, Or String)** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# For Loops - used to iterate over a sequence (like a list, tuple, dictiona
# for item in sequence:
# you can iterate over a list, string, tuple, dictionary, set, range() and

for x in reversed(range(1, 6)):  # Numbers from 1 to 5
    print(x)
print("Finished counting to 5!")

for x in range(1, 10, 2):  # Numbers from 1 to 9, step 2
    print(x)
print("Finished counting to 9!")

credit_card = "1234-5678-9012-3456"
for x in credit_card:
    if x == "-":
        continue  # Skip the hyphen
    print(x)

# Time module
print("Current time in seconds since epoch:", time.time())
local_time = time.localtime()
print("Local time:", local_time)

mytime = int(input("Enter time in seconds to countdown: "))
while mytime > 0:
    mins, secs = divmod(mytime, 60)
    timer = '{:02d}:{:02d}'.format(mins, secs)
    print(timer, end="\r")
    time.sleep(1)
    mytime -= 1

for x in range(0,mytime):
    time.sleep(1)
    print(x)
```

```
for x1 in range(mytime,0,-1):
    seconds = x1 % 60
    minutes = int ((x1/60)%60)
    hours = int((x1/3600)%24)
    print(f"{hours:02d}:{minutes:02d}:{seconds:02d}")
    time.sleep(1)

print("Time's up!")

# Nested loops - A loop inside another loop
for x in range(3):
    for y in range(1,10):
        print(y,end=' ')
    print()  # New line after inner loop

rows = int(input("Enter number of rows: ") )
colums = int(input("Enter number of colums: ") )
symbol = input("Enter a symbol to use: ")
for x in range(rows):
    for y in range(colums):
        print(symbol,end=' ')
    print()  # New line after inner loop
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_13_Collections___List__.md ---

🚀 Python Mastery Series - Topic 13: Collections - List, Tuple, Set, Dictionary! 🐍

Hey network! 👏 Today I'm sharing some sample code on **Collections - List, Tuple, Set, Dictionary** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Collections - List, Tuple, Set, Dictionary
# List = [] ordered and changeable. Duplicates OK
# Set ={} unordered and immutable, but add/remove OK. No Duplicates
# Tuple = () ordered and unchangeable. Duplicates OK. FASTER
# Dictionary = {key:value} unordered, changeable and indexed. No Duplicates

#List
fruits = ["apple", "banana", "cherry","coconut","orange"]
print(fruits)  # Output: ['apple', 'banana', 'cherry']
print(fruits[0])  # Output: apple
print(fruits[-1])  # Output: orange
print(fruits[1:3])  # Output: ['banana', 'cherry']
print(fruits[::2])  # Output: ['apple', 'cherry', 'orange']
print(fruits[::-1]) # Output: ['orange', 'coconut', 'cherry', 'banana', 'ap
fruits[0] = "kiwi"  # Change the value of the first item
print(fruits)  # Output: ['kiwi', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)
#print(dir(fruits))  # List of all attributes and methods of the list objec
#print(help(fruits))
print(len(fruits))  # Output: 5
print(max(fruits))  # Output: orange (based on alphabetical order)
print(min(fruits))  # Output: banana (based on alphabetical order)
print("banana" in fruits)  # Output: True
fruits.append("mango")  # Add an item to the end of the list
fruits.insert(1, "grape")  # Insert an item at a specific position
print(fruits)  # Output: ['kiwi', 'grape', 'banana', 'cherry', 'coconut', '
fruits.remove("banana")  # Remove an item by value
fruits.pop()  # Remove the last item
print(fruits)  # Output: ['kiwi', 'grape', 'cherry', 'coconut', 'orange']
fruits.sort()  # Sort the list in ascending order
print(fruits)  # Output: ['cherry', 'coconut', 'grape', 'kiwi', 'orange']
fruits.sort(reverse=True)  # Sort the list in descending order
print(fruits)  # Output: ['orange', 'kiwi', 'grape', 'coconut', 'cherry']
```

```
#fruits.clear()  # Remove all items from the list
print(fruits)  # Output: []

# Set ={} unordered and immutable, but add/remove OK. No Duplicates
fruits = {"apple", "banana", "cherry","coconut","orange"}
print(fruits)
print("banana" in fruits)  # Output: True
fruits.add("mango")  # Add an item to the set
fruits.remove("banana")  # Remove an item by value
fruits.discard("grape")  # Remove an item by value (no error if not found)
print(len(fruits))  # Output: 4
print(fruits)

# Tuple = () ordered and unchangeable. Duplicates OK. FASTER
fruits = ("apple", "banana", "cherry","coconut","orange")

print(len(fruits))
# print(help(fruits))
print(fruits.index("cherry"))  # Output: 2
print(fruits.count("cherry"))  # Output: 2
for fruit in fruits:
    print(fruit)
#print(fruits)

# Shopping Cart Application
foods = []
prices=[]
total=0
while True:
    food = input("Enter the item to add to the cart (or 'q' to quit): ")
    if food.lower() == 'q':
        break
    else:
        price = float(input(f"Enter the price of {food}: $"))
        foods.append(food)
        prices.append(price)

print("------------------------------YOUR CART-----------------------------
```

```
for food in foods:
    print(f"- {food}", end="")

for price in prices:
    print()

    print(f"- {price}", end="")



print(f"\nTotal price: ${total:.2f}")
total += price

# two dimensional list (list inside a list)

fruits = ("apple", "banana", "cherry","coconut","orange")
vegetables = ("carrot", "broccoli", "spinach","potato","onion")
meats = ("chicken", "beef", "pork","fish","lamb")
drinks = ("water", "soda", "juice","coffee","tea")
groceries = [fruits, vegetables, meats, drinks]
print(groceries)
print(groceries[0])  # Output: ('apple', 'banana', 'cherry', 'coconut', 'ora
print(groceries[1])  # Output: ('carrot', 'broccoli',   'spinach', 'potato'
print(groceries[2])  # Output: ('chicken', 'beef', 'pork', 'fish', 'lamb')
print(groceries[3])  # Output: ('water', 'soda', 'juice', 'coffee', 'tea')
print(groceries[0][0])  # Output: apple
print(groceries[1][1])  # Output: broccoli

for collection in groceries:
    print(collection)
    for item in collection:
        print(item)

# Print Phone Keypad
keypad = (
    ["1", "2", "3"],
    ["4", "5", "6"],
    ["7", "8", "9"],
    ["*", "0", "#"]
```

```
)
for row in keypad:
    for key in row:
        print(key, end=" ")
    print()  # New line after each row

# Python Quiz Application
questions = ("What is the capital of France?",
             "What is 2 + 2?",
             "What is the largest planet in our solar system?",
             "Who wrote 'To Kill a Mockingbird'?",
             "What is the chemical symbol for water?",)
options = ((),
        ("A. Paris", "B. London", "C. Berlin", "D. Madrid"),
        ("A. 3", "B. 4", "C. 5", "D. 6"),
        ("A. Jupiter", "B. Mars", "C. Earth", "D. Saturn"),
        ("A. Harper Lee", "B. Mark Twain", "C. Ernest Hemingway", "D. F. Sc
        ("A. H2O", "B. O2", "C. CO2", "D. N2") )
answers = ("A. Paris", "B. 4", "C. Jupiter", "D. Harper Lee", "E. H2O")
guesses = []
score = 0
question_num = 0

for question in questions:
    print ("----------------------------- QUESTION-----------------------
    print(question)
    for option in options[question_num]:
        print(option)
    guess = input("Enter (A, B, C, D or E): ").upper()
    guesses.append(guess)
    if guess == answers[question_num]:
        score += 1
        print("Correct!")
    else:
        print(f"Wrong! The correct answer is: {answers[question_num]}")
    question_num += 1

print("----------------------------- RESULTS ----------------------------
```

```
print("Answers: ", end="")
for answer in answers:
    print(answer, end=" ")
print()
print("Guesses: ", end="")
for guess in guesses:
    print(guess, end=" ")
print()
score_percentage = int((score / len(questions)) * 100)
print(f"Your score is: {score}/{len(questions)} ({score_percentage}%)")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_14_Dictionary___key_va.md ---

🚀 **Python Mastery Series - Topic 14: Dictionary = {Key:Value} Unordered, Changeable And Indexed. No Duplicates!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Dictionary = {Key:Value} Unordered, Changeable And Indexed. No Duplicates** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Dictionary = {key:value} unordered, changeable and indexed. No Duplicates
capital = {
    "USA": "Washington, D.C.",
    "India": "New Delhi",
    "China": "Beijing",
    "Russia": "Moscow",
    "France": "Paris"
}
print(dir(capital))  # List of all attributes and methods of the dictionary
```

```
# print(help(capital))  # Help on dictionary object
print(capital.get("India"))  # Output: New Delhi
print(capital["India"])  # Output: New Delhi
print(capital.get("Germany"))  # Output: Not Found (default value)
if capital.get("Germany"):
    print("Found")
else:
    print("Not Found")

capital.update({"Germany": "Berlin"})  # Add a new key-value pair
print (capital.get("Germany"))
capital.pop("Russia")  # Remove a key-value pair by key
print(capital)
capital.popitem()  # Remove the last inserted key-value pair
print(capital)
keys = capital.keys()  # Get all keys
values = capital.values()  # Get all values
for key in capital.keys():
    print(key)
    print(values)

# items = capital.items()  # Get all key-value pairs
for key, value in capital.items():
    print(f"The capital of {key} is {value}")

# Consession stand application
menu = {
    "Hot Dog": 3.50,
    "Popcorn": 2.50,
    "Soda": 1.50,
    "Candy": 1.00
}
cart = []
total = 0
print("Welcome to the Movie Theater Concession Stand!")
print("-----------------Menu:")
for key, value in menu.items():
    print(f"{key:10}: ${value:.2f}")
```

```
while True:
    item = input("Enter the item to add to the cart (or 'q' to quit): ")
    if item.lower() == 'q':
        break
    elif menu.get(item) is not None:
        cart.append(item)
        print("Item are added in the menu. Please try again.")
print()
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_15_Random_Numbers.md ---

🚀 **Python Mastery Series - Topic 15: Random Numbers!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Random Numbers** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Random Numbers

low =1;
high =100;

number = random.randint(low, high)
print(number)
options = ("Rock","Scissors","Paper")
option = random.choice(options)
print(option)

cards = ["2","3","4","5","6","7","8","9","10","A","J","Q","K"]
```

```
random.shuffle(cards)
print(cards)

# Python Number Guessing Game
low_num =1;
high_num =10;
answer = random.randint(low_num , high_num)
print(answer)

guesses =0
is_running = True
print("---------------------Python Number Guessing Game---")
print(f"Select a Number between {low_num} and {high_num}")

while is_running:
    guess  = input("Enter your Guess: ")
    if guess.isdigit():
        guess = int(guess)
        guesses += 1
        if guess <low_num or guess>high_num:
            print("Sorry...Out of Range")
            print(f"Please select a Number between {low_num} and {high_num}
        elif guess < answer:
            print("Too Low! Guess Again!")
        elif guess > answer:
            print("Too High! Guess Again!")
        else:
            print(f"You are Correct! The Answer is {answer}")
            print(f"Total No of Guess {guesses}")
            is_running = False
    else:
        print("Sorry....Guess accept by Numbers only!")
        print(f"Please select a Number between {low_num} and {high_num}")

# ⭐ rock, paper, scissors game 🪨

options = ("rock","paper","scissors")
running = True
```

```
while running:
    player = None
    computer = random.choice(options)
    while player is not options:
        player = input("Enter a choice like rock, paper, scissors game ")
        print(f"player Choice Is: {player}")
        print(f"Computer Choice Is:{computer}")

    if player == computer:
        print("Its Tie!")
    elif player =="rock" and computer == "scissors":
        print("You Win!")
    elif player =="paper" and computer == "rock":
        print("You Win!")
    elif player =="scissors" and computer == "paper":
        print("You Win!")
    else:
        print("You loser!")
    if not input("Play Again ?? ()y/n").lower =="y":
        running = False

print("Thanks for Playing!")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_16_Function___A_block_o.md ---

🚀 Python Mastery Series - Topic 16: Function - A Block Of Code Which Only Runs When It Is Called. You Can Pass Data, Known As Parameters, Into A Function.! 🐍

Hey network! 👏 Today I'm sharing some sample code on **Function - A Block Of Code Which Only Runs When It Is Called. You Can Pass Data, Known As Parameters, Into A Function.** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Function - A block of code which only runs when it is called. You can pas
#  A function can return data as a result.


def happy_birthday():
    print("Happy Birthday Dear Friend!")
    print()

happy_birthday()  # Function call
def greet_user(name):
    print(f"Hello, {name}!")
    print()

greet_user("Vinodh")  # Function call with argument
greet_user("Nanditha")  # Function call with argument


def display_invoice(name,amount,date):
    print("----- Invoice -----")
    print(f"Name: {name}")
    print(f"Amount: ${amount:.2f}")
    print(f"Date: {date}")
    print("-------------------")
    print()

display_invoice("Vinodh", 150.75, "2025-10-12")
display_invoice("Nanditha", 200.50, "2025-10-13")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_17_Returning_values_fro.md ---

🚀 **Python Mastery Series - Topic 17: Returning Values From Functions!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Returning Values From Functions** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Returning values from functions
def defadd(a, b):
    return a + b
c = defadd(5, 10)
print(c)  # Output: 15

# Default Arugments - A function that takes default arguments if no argumen
# 1. positional  2. keyword  3. default 4. arbitrary

def net_price(price, discount = 0.1, tax=0.05):
    return price * (1-discount)*(1+tax)

price1 = net_price(23)  # Using default discount and tax
print(f"Net price (default discount and tax): ${price1:.2f}")

price2 = net_price(23,0.0,10)  # Using default discount and tax
print(f"Net price (default discount and tax): ${price2:.2f}")

price3 = net_price(23,10)  # Using default discount and tax
print(f"Net price (default discount and tax): ${price3:.2f}")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_18_Keyword_arugments___.md ---

🚀 **Python Mastery Series - Topic 18: Keyword Arugments - A Function That Takes Keyword Arguments During The Function Call.!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Keyword Arugments - A Function That Takes Keyword Arguments During The Function Call.** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Keyword arugments - A function that takes keyword arguments during the fu
def hello_def(greeting,title,fname,lname):
    print(f"{greeting} {title} {fname} {lname}")

hello_def("Hello","Mr.","Vinodh","Kumar")
hello_def(title="Ms.",lname="Nanditha",fname="S",greeting="Hi")

for i in range(1,2):
    print(i,end="")
    print()

print("11","12", "13", "14", "15", sep="\n")
print("11","12", "13", "14", "15", sep="-")

# Arbitrary
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

--- Post_19_A_function_that_take.md ---

🚀 Python Mastery Series - Topic 19: A Function That Takes Arbitrary Number Of Arguments During The Function Call.! 🐍

Hey network! 👋 Today I'm sharing some sample code on **A Function That Takes Arbitrary Number Of Arguments During The Function Call.** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# A function that takes arbitrary number of arguments during the function ca
# *args - non-keyword arguments
# **kwargs - keyword arguments
def add_def(*args):
    total = 0
    for num in args:
        total += num
    return total

sum1 = add_def(1, 2, 3)
print(f"Sum (1, 2, 3): {sum1}")  #

def def_displayname(**args):
    for key, value in args.items():
        print(f"{key}: {value}")
        print()
def_displayname(fname="Vinodh", lname="Kumar", title="Mr.")
def_displayname(title="Ms.", lname="Nanditha", fname="S")

def def_address(**kwargs):
    for key in kwargs.keys():
        print(f"{key}")
        print()

def_displayname(street="123 Main St", city="Anytown", state="CA", zip="1234

def shipping_label(*args, **kwargs):
    print("Shipping Label:")
```

```
    for arg in args:
        print(arg,end ="")
    print()
    for value in kwargs.values():
        print(value,end ="")
    print()
    print("--------------------")
    print(f"{kwargs.get('street')  }, {kwargs.get('city')}, {kwargs.get('st
    print(f"Total items in shipping label: {len(args) + len(kwargs)}")

shipping_label("Hello!","Dr..","Vinodh Kumar", street="6 Gopinathan Gross S
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_20_Iterables_and_Iterat.md ---

🚀 **Python Mastery Series - Topic 20: Iterables And Iterators!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Iterables And Iterators** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
#Iterables and Iterators
# An iterable is an object that can be iterated (looped) over, such as a li
# An iterator is an object that represents a stream of data and can be itera
numbers = [1, 2, 3, 4, 5]  # This is an iterable (list)
for number in numbers:
    print(number, end = " - ")
print()
numbers1 = (1, 2, 3, 4, 5)  # This is an iterable (list)
for number1 in reversed(numbers1):
    print(number1, end = " - ")
```

```
print()
my_dec = {'a': 1, 'b': 2, 'c': 3}  # This is an iterable (dictionary)
for key in my_dec:
    print(f"{key}: {my_dec[key]}", end = " | ")
print()
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_21_Membership_Operators.md ---

🚀 **Python Mastery Series - Topic 21: Membership Operators!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Membership Operators** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
#Membership Operators
# The 'in' keyword is used to check if a value exists in a sequence (list, 
# The 'not in' keyword is used to check if a value does not exist in a sequ
word = "Python"
uinput = input("Enter a letter to check if it is in the word 'Python': ")
if uinput in word:
    print(f"'{uinput}' is in the word 'Python'")
else:
    print(f"'{uinput}' is not in the word 'Python'")

students = ["Alice", "Bob", "Charlie"]
student = input("Enter a student name to check if they are in the class: ")
if student not in students:
    print(f"{student} is not in the class.")
else:
    print(f"{student} is in the class.")
```

```
email = "vinodh@example.com"
if "@" in email and "." in email:
    print("Valid email address.")
else:
    print("Invalid email address.")

# List Comprehensions   - A concise way to create lists using a single line
# Expression  for item in iterable if condition
squares = [x**2 for x in range(1, 11)]
print(squares)  # Output: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
dobules = []
for i in range(1, 11):
    dobules.append(i * 2)
print(dobules)  # Output: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
grade = [85, 92, 78, 90, 88]
pass_grade = [g for g in grade if g >= 90]
print(pass_grade)  # Output: [85, 92, 90, 88]

# Match Case Statements - A control flow statement that allows you to match
# match variable:

def day_of_week(day):
    match day:
        case 1:
            return "Monday"
        case 2:
            return "Tuesday"
        case 3:
            return "Wednesday"
        case 4:
            return "Thursday"
        case 5:
            return "Friday"
        case 6:
            return "Saturday"
        case 7:
            return "Sunday"
```

```
        case _:
            return "Invalid day"

day = int(input("Enter a day number (1-7): "))
print(f"The day is: {day_of_week(day)}")

def is_weekend(day):
    match day:
        case "Saturday" | "Sunday":
            return True
        case "Monday" | "Tuesday" | "Wednesday" | "Thursday" | "Friday":
            return False
        case _:
            return "Invalid day"

day = input("Enter a day name (its Weekend ?): ")
print(f"The day is: {is_weekend(day)}")
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_22_Module_Creation___A_.md ---

🚀 **Python Mastery Series - Topic 22: Module Creation - A Module Is A File Containing Python Code That Can Define Functions, Classes, And Variables. It Allows You To Organize Your Code Into Reusable Components.!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Module Creation - A Module Is A File Containing Python Code That Can Define Functions, Classes, And Variables. It Allows You To Organize Your Code Into Reusable Components.** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Module Creation - A module is a file containing Python code that can defi
# To create a module, simply save your Python code in a file with a .py ext
#import math
print(math.sqrt(16))  # Output: 4.0
print(math.factorial(5))  # Output: 120
import math as m  # Importing with an alias
print(m.sqrt(16))  # Output: 4.0
print(m.factorial(5))  # Output: 120
from math import sqrt, factorial  # Importing specific functions
print(sqrt(16))  # Output: 4.0
print(factorial(5))  # Output: 120

import CoreModules  # Importing a custom module
result = CoreModules.pi
print(f"Value of pi: {result}")
result = CoreModules.square(5)
print(f"Value of square(5): {result}")
result = CoreModules.cube(3)
print(f"Value of pi: {result}")
result = CoreModules.factorial(6)
print(f"Value of pi: {result}")



# Variables scope - LEGB Rule
# Scope Resolution - LEGB Rule
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_23_L*Local*scope*e*ins.md ---

🚀 **Python Mastery Series - Topic 23: L - Local (Scope Inside A Function)!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **L - Local (Scope Inside A Function)** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# L - Local (scope inside a function)

def fun1():
    x=1
    print(x)

def fun2():
    x=2
    print(x)

fun1()
fun2()
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_24_E___Enclosing__scope.md ---

🚀 **Python Mastery Series - Topic 24: E - Enclosing (Scope Of Enclosing Functions)!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **E - Enclosing (Scope Of Enclosing Functions)** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# E - Enclosing (scope of enclosing functions)
def fun3():
```

```
    #x=4
    def fun4():
        x=4
        print(x)
    fun4()
    #print(x)

fun3()
# G - Global (module level scope)
x=10000
def fun5():
    #x=1
    print(x)

def fun6():
    #x=2
    print(x+1)

fun5()
fun6()
# B - Built-in (scope of built-in names)
import math as e
def fun7():
    #x=2
    print(e.sqrt(16))
fun7()

# if __name__ == "__main__": (this block will run only if the module is run
def main():
    print("This is the main function.")
    # You can add more code here that you want to run when the module is exe

if __name__ == "__main__":
    main()

# Python Banking Application
def show_balance(balance):
    print(f"Your current balance is: ${balance:.2f}")
```

```
def deposit(balance):
    amount = float(input("Enter the amount to deposit: $"))
    if amount <= 0:
        print("Deposit amount must be positive.")
        return 0
    return amount

def withdraw():
    amount = float(input("Enter the amount to withdraw: $"))
    if amount <= 0:
        print("Withdrawal amount must be positive.")
        return 0
    elif amount > balance:
        print("Insufficient funds.")
        return 0
    else:
        return amount

balance = 0.0
is_running = True
while is_running:
    print("\n--- Python Banking Application ---")
    print("1. Show Balance")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Exit")
    choice = input("Enter your choice (1-4): ")

    match choice:
        case "1":
            show_balance(balance)
        case "2":
            balance += deposit()
        case "3":
            balance -= withdraw()
        case "4":
            print("Thank you for using the Python Banking Application. Good
```

```
            is_running = False
        case _:
            print("Invalid choice. Please try again.")

# Python slot machines

def spin_slots():
    symbols = ["🍒","🍋","🍉","🔔","⭐"]
    results =[random.choice(symbols) for _ in range(3)]
    #for symbol in range(1):
    #    results.append(random.choice(symbols))
    #return results

def print_slots(row):
    print("|',{row}")

def getpayout(row, bet):
    if row[0]==row[1]==row==[2]:
        if row[0]=="🍒":
            return bet * 3
        elif row[0]=="🍋":
            return bet * 4
        elif row[0]=="🍉":
            return bet * 5
        elif row[0]=="🔔":
            return bet * 6
        elif row[0]=="⭐":
            return bet * 7
    return 0


def main():
    balance = 100
    print(f"Your starting balance is: ${balance:.2f}")
    print("Spinning the slots...")
    print("🍒🍋🍉🔔⭐")
    while balance > 0:
        print(f"Your starting balance is: ${balance:.2f}")
```

```
        bet = input("Enter your bet amount: $")
        if not bet.isdigit():
            print("Bet amount must be positive.")
            continue
        bet = int(bet )
        if bet <= 0:
            print("Bet amount must be positive.")
            continue
        elif bet > balance:
            print("Insufficient funds.")
            continue
        balance -= bet
        row = spin_slots()
        print("Spinning.....\n")
        print_slots(row)
    #spin_slots()
    #print_slots()
    payout = getpayout(row,bet)
    if(payout>0):
        print("You Won the Amt ${payout}")
        balance +=payout
    else:
        print("Better luck next time")

if __name__ == "__main__":
    main()



# Encryption program
chars = " " + string.punctuation + string.digits + string.ascii_letters
chars = list(chars)
key = chars.copy()
random.shuffle(key)
#print(f"chars:{chars}")
#print(f"key:{key}")

#Encrypt
plain_text = input("Enter the Message to encrypt:-")
```

```
cipher_text =""
for letter in plain_text:
    index = chars.index(letter)
    cipher_text += key[index]
print(f"Orginal Message Like : {plain_text}")
print(f"Encrypt Message Like : {cipher_text}")

#Decrypt
cipher_text = input("Enter the Message to decrypt:-")
plain_text =""
for letter in cipher_text:
    index = key.index(letter)
    plain_text += chars[index]
print(f"decrypt Message Like : {cipher_text}")
print(f"Orginal Message Like : {plain_text}")

# Hangman in Python
from wordlist import words
#words = ("apple","orange","banana","cocount","pineapple")
# dictonary key = ()
hangman_art = {0:(
        " ",
        " ",
        " "),
    1:(" o "
       "   "
       "   "),
    2:(" o  "
       " |  "
       "   "),
    3:(" o  "
       "/|  "
       "   "),
    4:(" o  "
       "/|\  "
       "   "),
    5:(" o  "
       "/|\  "
```

```
        "/   "),
    6:(" o  "
       "/|\  "
       "/-\  ")}

#print(hangman_art[6])

def display_man(wrong_guesses):
    for line in hangman_art[wrong_guesses]:
        print(line)

def display_hint(hint):
    print(" ".join(hint))

def display_answer(answer):
    print(" ".join(answer))

def main():
    answer = random.choice(words)
    hint = ["_"] * len(answer)
    wrong_guesses = 0
    guessed_letters = set()
    is_running = True
    while is_running:
        display_man(wrong_guesses)
        display_hint(hint)
        display_answer(answer)
        guess = input("Enter a Letter:").lower()
        if len(guess)!= 1 or not guess.isalpha():
            print("invalid Inputs")
            continue
        if guess in guessed_letters:
            print(f"{guess} is already guessed")
            continue
        guessed_letters.add(guess)
        if guess in answer:
            for i in range(len(answer)):
                if answer[i]==guess:
```

```
                    hint[i] = guess
        else:
            wrong_guesses +=1
        if "_" not in hint:
            display_man(wrong_guesses)
            display_answer(answer)
            print(" You Won!!!")
            is_running = False
        elif wrong_guesses >= (len(hangman_art) -1):
            display_man(wrong_guesses)
            display_answer(answer)
            print(" You Loosser !!!")
            is_running = False
if __name__ == "__main__":
    main()

# python object oriented programming
# Object - and Classes
class Car:
        def __init__(self, model, color, year): # Correct __init__ with arg
            self.model = model
            self.color = color
            self.year = year
        def drive(self):
            print(f"You Can drive the {self.model}")
        def stop(self):
            print(f"You Can stop the  {self.model}")
        def describe(self):
            print(f"You are driving {self.year} and {self.model} and color

# Now you can create a Car object with arguments
my_car = Car("Tesla Model 3", "Red", 2023)
print(my_car.year)
print(my_car.drive())
print(my_car.stop())
print(my_car.describe())

# Class Variables - shared among all instance of the class
```

```
# Define out side of the constructor
# Allow you to share data among all objects created from that classes
class Student:
    class_variables_year =2025
    class_variables_no_student = 15

    def __init__(self,name,age):
        self.name = name
        self.age = age
        Student.class_variables_no_student +=1

student1 = Student("Vinodh",45)
student2 = Student("Sona",45)

print(student1.name)
print(student2.name)
print(Student.class_variables_year)
print(Student.class_variables_no_student)

# Inheritance = Allows a class to inherit attributes from another clasess
# helps with code resuablity and extensiblity
# class child(Parent)
# Eg:- Animal - > Dog ->Cat -> Mouse
class Animals:
    speaks = ""
    def __init__(self,name):
        self.name = name
        self.is_alive = True

    def eat(self):
        print(f"{self.name} is Eating")
    def sleep(self):
        print(f"{self.name} is Sleeping")

class Dog(Animals):
    def speak(self):
        print("!!!SAPARK!!!")
```

```
class Cat(Animals):
    def speak(self):
        print("!!!MEOW!!!")

class Mouse(Animals):
    def speak(self):
        print("!!!SQUEEK!!!")

dog = Dog("Scooby")
cat = Cat("Rose")
mouse = Mouse("roosser")

print(cat.name)
print(cat.is_alive)
cat.eat()
cat.sleep()
cat.speak()

print(dog.name)
print(dog.is_alive)
dog.eat()
dog.sleep()
dog.speak()

print(mouse.name)
print(mouse.is_alive)
mouse.eat()
mouse.sleep()
mouse.speak()

# Multiple Inheritance - inherit from more than one parent class c(a,b)
# Multiple level Inheritance -inherit from a parent which inherits from ano
class Animal:
    def __init__(self,name):
        self.name = name
        print("Hello")
    def eating(self):
        print(f"{self.name} is eating")
```

```
    def sleeping(self):
        print(f" {self.name}  is Sleeping")
class Prey(Animal):
    def flee(self):
        print(f" {self.name}  is fleeing")
class Predator(Animal):
    def hunt(self):
        print(f" {self.name} is hunting")
class Rabbit(Prey):
    pass
class Hawk(Predator):
    pass
class Fish(Prey,Predator):
    pass

rabbit = Rabbit("Bugs")
hawk = Hawk("Tony")
fish = Fish("Nemo")
rabbit.flee()
hawk.hunt()
fish.flee()
fish.hunt()
rabbit.sleeping()

# Super () = Funtions used in a child class to call methods from a parent c
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_25_Allows_you_to_extend.md ---

🚀 Python Mastery Series - Topic 25: Allows You To Extend The Functionality Of The Inherited Methods! 🐍

Hey network! 👋 Today I'm sharing some sample code on **Allows You To Extend The Functionality Of The Inherited Methods** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Allows you to extend the functionality of the inherited methods
class Shape:
    def __init__(self,color,is_filled):
        self.color = color
        self.is_filled = is_filled
    def describe(self):
        print(f"It is {self.color} and {'filled' if self.is_filled else 'no

class Circle(Shape):
    def __init__(self,color,is_filled,radius):
        super().__init__(color,is_filled)
        self.radius = radius
    def describe(self):
        print(f"It is a circle wiht {self.color} and Radious of the circle
        super().describe()
class Square(Shape):
    def __init__(self,color,is_filled,width):
        super().__init__(color,is_filled)
        self.width = width
    def describe(self):
        print(f"It is a square with {self.color} and Radious of the circle
        super().describe()
class Triangle(Shape):
    def __init__(self,width,color,is_filled,height):
        super().__init__(color,is_filled)
        self.width = width
        self.height = height
    def describe(self):
        print(f"It is a Triangle with {self.color} and Radious of the circl
        super().describe()
```

```
circle = Circle("Yellow",True,10)
square = Square("Green",False,1)
triangle = Triangle("RED",False,6,6)
print(circle.color, circle.is_filled , circle.radius)
circle.describe()
print(square.color, square.is_filled , square.width)
square.describe()
print(triangle.color, triangle.is_filled , triangle.width, triangle.height)
triangle.describe()

# Polymorphism - Have Many forms or faces
#1.Inheritance - An object could be treated of the same types a parent clas
#2."Duck typing" - Object must have necessary attributes/methods
from abc import ABC, abstractmethod
class Shape:
    @abstractmethod
    def area(self):
        pass
class Circle(Shape):
    def __init__(self,radius):
        self.radius = radius
    def area(self):
        return 3.14*self.radius**2

class Square(Shape):
    def __init__(self,side):
        self.side = side
    def area(self):
        return self.side**2
class Triangle(Shape):
    def __init__(self,base,height):
        self.base = base
        self.height = height
    def area(self):
        return self.base*self.height
class Pizza(Circle):
    def __init__(self,topping,radius):
        self.topping = topping
```

```
        super().__init__(radius)
        #self.radius = radius

shapes = [Circle(4),Square(5),Triangle(5,5),Pizza("CheessePizza",15)]
for shape in shapes:
    print(f"{shape.area()}  cm2")

# Duck Typing = Another way to achive polymorphism besides Inheritance
# Object must have the minimum necessary attributes and methods
# If it looks like DUCK and quacks like a duck, it must be a duck

class Animal:
    alive = True

class Dog(Animal):
    def speak(self):
        print("WOOFF!!!")
class Cat(Animal):
    def speak(self):
        print("MEOW!!!")
class Car():
    alive = False
    def speak(self):
        print("HAUNK!!!!!")

animals=[Dog(),Cat(),Car()]

for animal in animals:
    animal.speak()
    print(animal.alive)

# Static Method -A Method that belong to a class rather than any object fro
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_26_Usually_used_for_gen.md ---

🚀 **Python Mastery Series - Topic 26: Usually Used For General Utility Functions.!** 🐍

Hey network! 👋 Today I'm sharing some sample code on **Usually Used For General Utility Functions.** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Usually used for general utility functions.

# Instance method - best for operations on iteration of the class(objects)
# Static method -  best utility functions that do not need access to class (
class Employee:
    def __init__(self,name,position):
        self.name = name
        self.position = position
    def get_info(self):
        return f"{self.name} = {self.position}"
    @staticmethod
    def is_vaild_position(position):
        vaild_position = ["Manager", "Software","Cook"]
        return position in vaild_position

print(Employee.is_vaild_position("Manager"))
employee1 = Employee("Vinodh", "Software")
employee2 = Employee("Raja", "Manager")
employee3 = Employee("Nani", "Cook")

print(employee1.get_info())
print(employee2.get_info())
print(employee3.get_info())
```

```
#Class Method - Allow operations related to the class itself
                #Take (cls) as the first parameter , which reprents the cla
                #
# Instance Method - Best for operation on instance of the class(objects)
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---

--- Post_27_Static_Method___Best.md ---

🚀 **Python Mastery Series - Topic 27: Static Method - Best For Utility Functions That Do Not Need Access To Class Data!** 🐍

Hey network! 👏 Today I'm sharing some sample code on **Static Method - Best For Utility Functions That Do Not Need Access To Class Data** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Static Method - Best for utility functions that do not need access to cla
# Class Method - Best for class_level data or require to the class itself

class Student:
    count = 0
    total_gpa = 0
    def __init__(self,name,gpa):
        self.name = name
        self.gpa = gpa
        Student.count += 1
        Student.total_gpa += gpa

# Instance method
    def get_info(self):
```

```
        return f"{self.name} {self.gpa}"

    @classmethod
    def get_count(cls):
        return f"Total No of Students:{cls.count}"

    @classmethod
    def get_avg_gpa(cls):
        if cls.count == 0:
            return 0
        else:
            return f"Average GPA : - {cls.total_gpa/cls.count:.2f}"

Student("spongebob","2.8")
Student("Vinodh","3")
Student("Charu","4.5")
Student("nandthiya","4.8")

print(Student.get_count())
print(Student.get_avg_gpa())

# Magic Method - Dunder methods (double underscore) __init__,__str__,__eq__
# They are automatically called by many of the python built in operations
# They allow developers to define or customize  the behavior of objects

class Book:
    def __init__(self,title,autor,no_pages):
        self.title = title
        self.autor = autor
        self.no_pages = no_pages

    def __str__(self):
        return f"'{self.title}' by {self.autor}"

    def __it__(self,other):
        return self.no_pages == other.no_pages

    def __eq__(self,other):
```

```
        return self.title == other.title and self.autor == other.autor

    def __gt__(self,other):
        return self.no_pages >= other.no_pages

    def __add__(self,other):
        return self.no_pages + other.no_pages

    def __contains__(self, keywords):
        return keywords in self.title or keywords in self.autor

    def __additem__(self,key):
        if key == "title":
            return self.title
        if key == "autor":
            return self.autor


b1= Book("AAA","Mr.Roo",422)
b2= Book("BBB","Mr.Roo",422)
b3= Book("CCC","Mr.Roo",422)

print(b1)
print(b1 == b2)
print(b2 > b3)
print(b2 + b3)
print("s" in b3)
print(b3['autor'])
print(b3['title'])

# @Property - Decorator used to define a method as a property (it can be ac
# Benefit : - add addtional logic when read , write, delete attributes
# Gives you getter ,setter and deleter method
#
class Rectangle:
    def __init__(self, width,height):
        self._width = width
        self._height = height
```

```
    @property
    def width(self):
        return f"{self._width:.1f} cms"

    @property
    def height(self):
        return f"{self._height:.1f} cms"

    @width.setter
    def width(self,new_width):
        if new_width >0:
            self._width = new_width
        else:
            print("Width must be greater then zero")

    @height.setter
    def height(self,new_height):
        if new_height >0:
            self._height = new_height
        else:
            print("Height must be greater then zero")

    @width.deleter
    def width(self):
        del self._width
        print("Width info are deleted")

    @height.deleter
    def height(self):
        del self._height
        print("Height info are deleted")

rec = Rectangle(3,4)
rec.width = 999
rec.height = 1000

#print(rec.width)
```

```
#print(rec.height)

del rec.width
del rec.height

# @ Decorders
def add_sprinkles(func):
    def wrapper(*args, **kwargs):
        func(*args, **kwargs)
        print("Add your MOJ")
    return wrapper

def add_fudge(func):
    def wrapper(*args, **kwargs):
        print("*************************")
        func(*args, **kwargs)
    return wrapper
@add_fudge
@add_sprinkles
def get_ice_cream(flavor):
    print("Here is your {flavor} Ice Cream")

get_ice_cream("VANILLA")

# Exceptions - An  event that interrupts the flow of a program
#(ZeroDivisionError,TypeError,ValueError)
# Try,Catch and finally



try:
    # Try some Code
    number = int(input("Enter Your No:- "))
    print(1/number)
except ZeroDivisionError:
    # Handle Expection
    print("You Cann't divide by Zero")
except ValueError:
    # Handle Expection
```

```
    print("You Can enter number only!!!")
except Exception:
    # Handle Expection
    print("Error is here")
finally:
    print("Post work to be handle!!!")

try:
    # Try some Code
except ZeroDivisionError:
    # Handle Expection
except ValueError:
    # Handle Expection
except Exception:
    # Handle Expection
finally:

# Python File detection
file_path = "D:\\Pythonbase\\PythonProjects\\test.txt"
file_dir = "D:\\Pythonbase\\PythonProjects"
if os.path.exists(file_path):
    print("Allowed")
    if os.path.isfile(file_path):
        print("Yaaaaaaa")
    else:
        print("Noooooo")
elif os.path.isdir(file_dir):
    print("Ya..Floder")
else:
    print("Not Allowed")

# Python writing files like .txt,.json,.csv
employee = ["Vinodh","Charuhetuki","Nanditha","Kalaimathi"]
employee1 = {
    "name": "Vinodh",
    "age":"45",
    "job":"Software Mobile Developer"
}
```

```
empcsv = [["Name", "Age", "Job"],
          ["Vinodh","45","UnEmpolyment"],
          ["Charuhetuki","12","Student"],
          ["Nanditha","7","Student"]]
txt_data = "Writing thee files using python functions"
file_dir = "D:\\Pythonbase\\PythonProjects\\"
txtfilepath = file_dir +"output.txt"
jsonfilepath = file_dir +"output.json"
csvfilepath = file_dir +"output.csv"

#with open(file= txtfilepath, mode = "w") as file:
#    file.write(txt_data)
#    print("Done")

try:
    with open(file= txtfilepath, mode = "a") as file:
        for emp in employee:
            file.write("\n"+ emp)
        print("Done")
except FileExistsError:
    print("File Already Avilables!!!")

try:
    with open(file= jsonfilepath, mode = "a") as file:
        json.dump(employee1,file)
        print("Done")
except FileExistsError:
    print("File Already Avilables!!!")

try:
    with open(file= csvfilepath, mode = "a", newline ="") as file:
        writer = csv.writer(file)
        for row in empcsv:
            writer.writerow(row)
        print("Done")
except FileExistsError:
    print("File Already Avilables!!!")
```

```
file_dir = "D:\\Pythonbase\\PythonProjects\\"
txtfilepath = file_dir +"output.txt"
jsonfilepath = file_dir +"output.json"
csvfilepath = file_dir +"output.csv"

with open(txtfilepath,"r")as file:
    content = file.read()
    print(content)

with open(jsonfilepath,"r")as file:
    content = json.load(file)
    #for linejson in content:
    print(content)

with open(jsonfilepath,"r")as file:
    content = csv.reader(file)
    for csvline in content:
        print(csvline)

date = datetime.date(2025,1,11)
print(date)

today = datetime.date.today()
print(today)
time1 = datetime.time(12,30,0)
print(time1)
now1 = datetime.datetime.now()
print(now1)
now2 = now1.strftime("%H:%M:%S %d-%m-%y")
print(now2)

targettime = datetime.datetime(2030,1,2,12,30,1)
curgettime = datetime.datetime.now()
if targettime < curgettime :
    print("Target data has passed!!!")
else:
    print("Target data not passed!!!")
```

```
def set_alarm_time(alarm_time):
    print(f"Alaram set for {alarm_time}")
    sound_file = "Kandra_Music.mpeg"
    is_running = True

    while is_running:
        currttimenow = datetime.datetime.now().strftime("%H:%M:%S")
        print(currttimenow)
        if currttimenow ==alarm_time:
            print("Wake UP!!!")
            pygame.mixer.init()
            pygame.mixer.music.load(sound_file)
            pygame.mixer.music.play()
            while  pygame.mixer.music.get_busy():
                time.sleep(1)

            is_running = False
        time.sleep(1)

if __name__=="__main__":
    alarm_time = input("Enter the Time(HH:MM:SS):")
    set_alarm_time(alarm_time)
    print("Done!!!")

# Multithreading = used to perform multiple taks concurrently (Multitasking
# Good for I/OBound tasks like reading files or fetching data from APIs
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

--- Post_28_Threading_Thread_Tar.md ---

🚀 Python Mastery Series - Topic 28: Threading.Thread(Target=My_Function)! 🐍

Hey network! 👋 Today I'm sharing some sample code on **Threading.Thread(Target=My_Function)** in Python. Whether you're a beginner or just brushing up your basics, I hope you find this useful! 👇

```
# Threading.Thread(Target=my_function)
def walk_dog(first):
    time.sleep(8)
    print(f"You Finished walking the {first } Dog!!!")

def take_out_trash():
    time.sleep(2)
    print(f"You Finished out the Trash cleaup!!!")

def get_mail():
    time.sleep(2)
    print(f"You Got the Mail!!!")

#walk_dog()
#take_out_trash()
#get_mail()

chore1 = threading.Thread(target=walk_dog("Scooby"))
chore1.start()
chore2 = threading.Thread(target=take_out_trash)
chore2.start()
chore3 = threading.Thread(target=get_mail)
chore3.start()

chore1.join()
chore2.join()
chore3.join()
print("All chore Jbs are completed")

# Connect PokeAPI using Python
```

```
base_url = "https://pokeapi.co/api/v2/"

def get_pokeman_info(name):
    url = f"{base_url}/pokemon/{name}"
    response = requests.get(url)
    print(response)
    if response.status_code ==200:
        print("Processed the Request APIs!!!")
        pokeman_name = response.json()
        return pokeman_name
    else:
        print("Not Processed the Request APIs!!!")

pokeman_name = "pikachu"
pokeman_info = get_pokeman_info(pokeman_name)
if pokeman_info:
    pass
    #print(f"{pokeman_info["name"]}")
    #print(f"{pokeman_info["order"]}")

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Om Siva")
        self.setGeometry(700,150,500,500)
        self.setWindowIcon(QIcon("OmSivaImage.jpeg"))

def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec_())

if __name__=="__main__":
    main()
```

```
# End of the program
print("Thank you for using the Python program. Goodbye!")

#Sum the numbers in the list
numbers = [1,2,3,4,5,6,7,8,9,10]
sum = 0
for number in numbers:
    sum += number
print(sum)
```

What are your favorite tips regarding this topic? Let me know in the comments! 💬

Python #Coding #Programming #Developer #Learning #Tech #PythonProgramming

---